0212. Word Search II

Hard · Tries

leetcode.com/problems/word-search-ii

Problem

- Given a letter grid and a list of words, return all words that can be formed by adjacent (up/down/left/right) cells, each cell used once per word.

Intuition

- Running Word Search (problem 79) once per word repeats the same grid exploration.
- Put all words in a trie and DFS the grid once from each cell, walking the trie in parallel; abandon a path as soon as no word has that prefix.

Brute Force

- For each word run a backtracking search: O(W m n 4^L).

Optimal Approach

- Build a trie; store the full word at its end node.
- DFS(r, c, node): child = node.next[board[r][c]]; if null return.
- If child.word != null, record it and clear it (avoid duplicates).
- Mark the cell '#', recurse 4 directions, restore the cell.
- Optional pruning: remove trie leaves once their words are found.

Worked Example

```
board rows: oaan / etae / ihkr / iflv, words [oath, pea, eat, rain].

start (0,0) 'o' -> (0,1) 'a' -> (1,1) 't' -> (2,1) 'h' : found "oath"
start (1,3) 'e' -> (1,2) 'a' -> (1,1) 't'                : found "eat"
"pea", "rain" have no path -> result [oath, eat]
```

Java Solution

```java
public class WordSearchII {
    private static class Node {
        Node[] next = new Node[26];
        String word;
    }

    public List<String> findWords(char[][] board, String[] words) {
        Node root = new Node();
        for (String w : words) {
            Node n = root;
            for (char c : w.toCharArray()) {
                if (n.next[c - 'a'] == null) n.next[c - 'a'] = new Node();
                n = n.next[c - 'a'];
            }
            n.word = w;
        }
        List<String> res = new ArrayList<>();
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++) dfs(board, r, c, root, res);
        return res;
    }

    private void dfs(char[][] b, int r, int c, Node parent, List<String> res) {
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || b[r][c] == '#') return;
        char ch = b[r][c];
        Node n = parent.next[ch - 'a'];
        if (n == null) return;
        if (n.word != null) { res.add(n.word); n.word = null; }
        b[r][c] = '#';
        dfs(b, r + 1, c, n, res);
        dfs(b, r - 1, c, n, res);
        dfs(b, r, c + 1, n, res);
        dfs(b, r, c - 1, n, res);
        b[r][c] = ch;
    }
```

Java Solution (2)

```
}
```

Complexity

- Time: O(m n 4 3^(L-1)) worst case, with heavy pruning in practice. Space: O(total word chars).

Pitfalls

- Clear found words to avoid duplicate results.
- Restore the board cell after recursion.